Agent State in Agentic AI Systems

A Practitioner's Framework for Working Memory Design, Logging Strategy, and Multi-Agent Coordination

Author: Clarence "Professor Bone" Downs
Affiliation: Professor Bone Lab
Program: Johns Hopkins University | Agentic AI Certificate Program
Date: December 2025 | Revised February 2026

Abstract

This paper develops a practitioner-oriented framework for agent state management in large language model (LLM)-based agentic systems. The theoretical foundation is the Cognitive Architectures for Language Agents (CoALA) framework (Sumers et al., 2024). This work extends that foundation with actionable engineering guidance for state schema design, logging strategy, and production deployment.

There is a failure mode that motivates the most significant contribution of this edition. An agent fails to act on information that was technically present in its context. The retrieval succeeded. The data was available. The agent behaved as though it had seen none of it. This is not a retrieval failure. It is a gating failure, and the standard remediation of adding more memory addresses the wrong layer entirely.

The paper's primary contributions are: (1) an operational interpretation of CoALA's working memory concept for software engineering contexts; (2) a novel taxonomy of four state update classes with corresponding logging policies; (3) practical design patterns for single-agent and multi-agent state coordination; and (4) a structural model distinguishing between information that is present in state and information that is actively shaping behavior, a distinction with direct consequences for how engineers design, instrument, and debug production systems.

The fourth contribution connects to a companion theoretical paper, *The Cognitive Horizon of an Agent* (Downs, 2026), which develops the formal model underlying Section 3.5. The two documents are intended to be read together.

Keywords: agent state, working memory, agentic AI, CoALA, multi-agent systems, LangGraph, observability, state management, cognitive horizon, operative state, gating failures

---

1. Introduction

Agent state management is not an implementation detail. It is an architectural decision that determines what a system can reason about, how it can be debugged, and whether it behaves reliably under operational conditions.

This paper addresses the gap between the theoretical frameworks that define what agent state should be and the engineering decisions practitioners face when building production systems. CoALA (Sumers et al., 2024) provides the strongest theoretical foundation available, drawing on decades of cognitive architecture research to establish working memory, long-term memory taxonomies, and decision-making cycles as central constructs. What CoALA does not fully address is the engineering layer: how to design schemas before implementation begins, what to log and what to discard, how to validate state

across multi-step workflows, and how to coordinate state across multiple agents without introducing fragility.

This paper addresses those questions directly.

It also addresses a fifth question that the original edition did not anticipate: why an agent sometimes fails to act on information that was clearly available to it. The answer is not always retrieval failure. Often, the information was retrieved. It was present. Something in the architecture kept it from becoming operative. Section 3.5 develops the structural model for understanding and diagnosing this class of failure.

1.1 Scope and Positioning

This work is positioned as applied research bridging theory and practice. It interprets and extends existing theory, primarily CoALA, for engineering contexts. Novel contributions, including the logging taxonomy and the latent/operative state distinction, are proposed practitioner frameworks requiring empirical validation rather than established scientific results.

The companion paper, *The Cognitive Horizon of an Agent* (Downs, 2026), develops the formal model of operative cognition applied in Section 3.5. Practitioners seeking deeper theoretical grounding are encouraged to read both documents together.

---

2. Theoretical Foundations

2.1 Classical Cognitive Architectures

The challenge of managing state in intelligent systems predates LLMs by decades. The cognitive architecture literature offers foundational insights that remain directly applicable to contemporary agent design.

SOAR (State, Operator, And Result), developed by Laird, Newell, and Rosenbloom (1987) and refined over subsequent decades (Laird, 2012; Laird, 2022), models cognition through a production system operating over working memory. SOAR distinguishes between working memory, the active substrate maintaining situational awareness across perceptual input, intermediate reasoning, active goals, and long-term memory buffers, and long-term memories that include semantic, episodic, and procedural knowledge persisting across tasks.

Critically, in SOAR, working memory is not a buffer. It is the space where reasoning occurs. All interactions with the environment flow through structures attached to the top state. State changes are the primary mechanism through which cognition advances (Laird, 2022).

ACT-R (Adaptive Control of Thought, Rational), developed by Anderson and colleagues (Anderson, 1996; Anderson & Lebiere, 1998), presents a complementary modular architecture. ACT-R's production system operates over buffer contents that mediate between specialized modules. The buffer-mediated approach carries a key implication for LLM agent design: state should be structured, explicitly defined, and serve as the sole interface through which reasoning proceeds.

A systematic review of 84 cognitive architectures over 40 years (Kotseruba & Tsotsos, 2020) finds that successful architectures, despite their diversity, share common principles: state must be explicit rather than implicit, structured rather than amorphous, and designed to support both current task execution and meta-level reasoning about the task itself.

2.2 The CoALA Framework

The Cognitive Architectures for Language Agents framework (Sumers et al., 2024), published in *Transactions on Machine Learning Research*, provides the theoretical foundation for this paper. CoALA proposes that a language agent can be understood through three primary components.

Memory System: Working memory provides short-term storage for information relevant to the current decision cycle, functioning as a scratchpad that persists

across LLM calls. Long-term memory provides persistent storage across semantic, episodic, and procedural dimensions.

Action Space: External actions interact with the environment through tool use, API calls, and similar mechanisms. Internal actions operate on the agent's own memory through reasoning, retrieval, and learning.

Decision-Making Cycle: A structured loop alternating between planning and execution governs agent behavior.

CoALA's central insight is that LLMs function analogously to production systems in classical architectures. They define distributions over changes to text, just as productions define possible string modifications. Control structures developed for production systems may therefore apply to LLM-based agents.

This paper adopts CoALA's conceptual framework and develops operational guidance for implementing it in software systems. Where CoALA asks what components a language agent should have, this paper asks how engineers should implement those components in production code, and extends CoALA's working memory model in Section 3.5 to distinguish between state presence and state operability.

2.3 The ReAct Paradigm

The ReAct paradigm (Yao et al., 2023), presented at ICLR 2023, demonstrates the practical value of interleaving reasoning and action in LLM agents. Agents generating both reasoning traces and task-specific actions outperform those doing either in isolation.

ReAct illustrates a principle that CoALA theorizes explicitly: state representation is necessary to carry forward results across thought-action-observation cycles. The agent must track what has been tried, what information has been gathered, and what remains to be done. Well-managed working memory enables more effective agent behavior. Poorly managed working memory produces agents that repeat earlier steps, lose context mid-task, and fail to learn from within-task experience.

2.4 LangGraph and Contemporary Orchestration

LangGraph (LangChain, 2024) operationalizes these theoretical insights through a graph-based orchestration framework that treats state as a first-class architectural component. Applications are represented as directed graphs where nodes are processing steps and edges define transitions, with explicit state flowing through the graph.

Concept	Description
StateGraph	Container modeling the application as nodes and edges
TypedDict State	Strongly-typed state schemas defining information flow
Checkpointers	Mechanisms persisting state for recovery and human-in-the-loop
Reducers	Functions defining how concurrent state updates merge

LangGraph's StateGraph implements the decision-making cycle. TypedDict schemas structure working memory. Checkpointers enable persistence analogous to long-term memory formation. The alignment between LangGraph's design and CoALA's framework is not coincidental.

2.5 Memory in Generative Agents

Park et al. (2023) demonstrated sophisticated memory management in simulated social agents through a memory stream architecture that logs experiences continuously in natural language, retrieves relevant memories based on recency, importance, and relevance, and periodically synthesizes memories into higher-level reflections.

This work illustrates a distinction that this paper extends formally: the difference between logging everything and using information effectively. The memory stream is comprehensive but not directly operative; it requires retrieval and attention mechanisms to become actionable working memory. That distinction, between comprehensive recording and selective activation, informs both the logging taxonomy in Section 5 and the latent/operative state model in Section 3.5.

3. Defining Agent State: An Operational Interpretation

3.1 From Theory to Implementation

CoALA defines working memory as information "relevant to the current decision cycle" that "persists across LLM calls" and functions as "a scratch pad that can be used for reasoning" (Sumers et al., 2024). This paper operationalizes that definition for software engineering contexts:

> **Agent state** is a structured, typed data object representing the information an agent currently has access to for completing its task. It is explicitly defined by a schema, mutable across reasoning steps, and serves as the sole interface through which the LLM reasons about the task.

Property	Theoretical Basis (CoALA)	Implementation Implication
Structured	Working memory as organized scratchpad	Use TypedDict, dataclass, or Pydantic schemas
Typed	Distinct memory components	Explicit type annotations for all fields
Mutable	State changes across decision cycles	Design for update patterns, not just initialization
Bounded	Finite working memory capacity	Include only task-relevant information
Observable	Enables reasoning and debugging	State must be serializable and inspectable

3.2 What Agent State Contains

Drawing from CoALA's working memory concept, agent state typically includes five categories of information.

Task Context holds the goal or objective for the current task, constraints, and user preferences relevant to task execution.

Progress Tracking holds steps completed with their outcomes, the current phase of execution, and remaining steps or open questions.

Retrieved Information holds documents, data, or facts gathered during execution, tool outputs, and relevant context from long-term memory.

Decision History holds key decisions and their rationale, branch points taken in conditional logic, and errors encountered with recovery actions.

Intermediate Results holds partial outputs under construction, drafts and analyses in progress, and hypotheses under consideration.

3.3 What Agent State Is Not

Agent state is not conversation history. Raw message logs are not structured for reasoning. State extracts and organizes relevant information from conversation rather than preserving the conversation itself.

Agent state is not the schema definition. The schema is the blueprint; state is the runtime instance with actual values. This parallels the class/instance distinction in object-oriented programming.

Agent state is not LLM internal representation. LLMs have no persistent internal state across API calls. What appears as memory in LLM conversations is context window content passed explicitly in each request.

Agent state is not long-term memory. Following CoALA's distinction, long-term memory persists across tasks and sessions. Working memory is task-scoped and potentially ephemeral.

Agent state is not logging. Logs observe state. They do not define it. This distinction is developed in Section 4.

3.4 The Primacy of State in Agent Reasoning

A key insight from cognitive architectures, formalized in CoALA, is that working memory defines the boundary of what an agent can reason about. This paper articulates it as a design principle:

> **Design Principle (State Primacy):** Information not represented in agent state is not available for agent reasoning. The state schema defines the agent's cognitive horizon for the current task.

This principle synthesizes insights from CoALA and classical cognitive architectures. It is proposed as a design heuristic rather than an established theoretical result. Formal development of the Cognitive Horizon concept appears in the companion paper (Downs, 2026).

The practical implications are direct. If retrieved documents are not stored in state, the agent cannot reference them. If previous decisions are not tracked, the agent cannot explain its reasoning. If error history is not maintained, the agent cannot learn from within-task failures. If progress is not marked, the agent cannot know where it stands in execution.

State schema design is an architectural decision, not an implementation detail.

3.5 From State Presence to Operative Cognition

This section extends the framework developed in Sections 3.1 through 3.4. It draws on the formal model introduced in The Cognitive Horizon of an Agent (Downs, 2026) and applies that model to practical engineering decisions.

Section 3.4 establishes a foundational principle: information not in state cannot influence reasoning. That claim is correct. Production experience reveals that it is not sufficient.

Consider a concrete failure. An agent retrieves a relevant document. The retrieval logs confirm success. The document enters the context window. The agent surfaces it during intermediate reasoning. When the decision point arrives, the agent acts as though it had never seen the document.

Engineers reach for the obvious remediation: better retrieval, larger context windows, more memory. These interventions sometimes help. More often they do not, because the failure was not at retrieval. The information was present. Something in the architecture kept it from becoming operative.

That something has a name. Understanding it changes how we build.

3.5.1 The Refined Model

Cognition at time t is not merely bounded by state. It is governed by three interacting factors:

```
C(t) ≈ P(t)( A(t)( S(t) ) )
```

Where:

- **S(t)**, State: defines what information is feasible, the outer boundary
- **A(t)**, Attention: determines what is salient, what the agent actually weights
- **P(t)**, Policy: determines what is permitted to influence action, the governance layer

This formulation is nested, not additive. Attention operates on state. Policy operates on the output of attention. Each layer is a filter. Information that passes the first filter can still be suppressed by the second.

The state-primacy principle describes the S(t) layer. That layer is real and important. But two more layers above it determine whether information in state ever reaches behavior.

3.5.2 Latent vs. Operative State

Within S(t), within the active state, not all information behaves the same way.

Latent state refers to information present in the context but not actively shaping planning, tool selection, or decisions. The information has been retrieved. It is in the context window. It is not working.

Operative state refers to the subset of state that is simultaneously salient to the attention mechanism and permitted by the policy layer to influence action. This is what the agent is actually reasoning from.

The transition from latent to operative is where most production failures live. Standard instrumentation stops at retrieval. It confirms the memory was fetched, registers success, and moves on. What it does not confirm is whether the information ever crossed from latent into operative. That second crossing is where the failure usually occurred.

3.5.3 Gating Failures: A Structural Taxonomy

When information is latent rather than operative, the cause is structural. These are gating failures: the agent possesses the knowledge but is architecturally prevented from using it.

Four structural causes appear most frequently in production systems.

Attention suppression. A relevant signal is present in state but ranked too low by the attention mechanism to influence output. High-frequency or high-recency signals crowd it out. The agent is not ignoring the signal deliberately; it never reaches the threshold of salience.

Policy blocking. The information is salient but constrained by governance rules, cost functions, or behavioral guardrails that prevent the agent from acting on what it knows. When policy is well-calibrated, this suppression is intentional and correct. When policy is outdated or miscalibrated, it silently suppresses valid reasoning without producing any visible error signal.

Spurious signal dominance. Attention is captured by noise. Irrelevant but high-weighted signals crowd out what matters. The agent is reasoning well, but over the wrong subset of its state.

Policy-state misalignment. The policy was designed for a prior state configuration. State has since evolved, but policy has not been updated. The agent applies correct rules to an incorrect model of the current situation.

In every case, the system possesses the knowledge. The failure is in the gating. The fix is not more data. It is better gating design.

3.5.4 The Three-Question Diagnostic Protocol

When an agent fails to act on available information, three questions asked in sequence locate the failure layer.

Question 1: Was the information retrieved into state?
If no, this is a retrieval failure. The information never crossed the outer boundary of the Cognitive Horizon. Fix the retrieval mechanism.

Question 2: Was the information made salient?
If yes to Question 1 but no to Question 2, this is an attention failure. The information was latent. It was not weighted appropriately by the attention mechanism. Fix the salience signals or attention weighting.

Question 3: Was the information permitted by policy to influence action?
If yes to Questions 1 and 2 but no to Question 3, this is a policy failure. The information was salient and present, but governance constraints blocked it. Fix the policy calibration, or escalate the policy review.

If all three questions answer yes and the agent still failed, that is a reasoning failure, and a different investigation entirely.

This protocol reframes agent evaluation from memory accuracy to structural operability. The diagnostic question is not "what did the agent know?" but "what was the agent structurally capable of using?" That is a more precise question. It leads to more precise interventions.

3.5.5 Engineering Implications

This model reshapes several practical decisions.

Schema design. The four-category logging taxonomy in Section 5 distinguishes information by persistence and observability requirements. The latent/operative distinction adds a question to schema design: is this field likely to be used, or merely present? Fields that are consistently retrieved but rarely acted upon are candidates for gating investigation, not schema expansion.

Instrumentation. Standard observability confirms that information entered state. Operative-state instrumentation asks whether that information influenced a downstream decision. The gap between those two measurements is the gating failure surface. Logging which state fields were referenced at each decision point, rather than just which fields were retrieved, converts invisible failures into measurable ones.

Debugging. The three-question protocol in Section 3.5.4 provides structured entry points into failure investigation. Applied systematically, it prevents engineers from investing in retrieval improvements when the actual failure is policy miscalibration, or in policy revisions when the actual failure is attention weighting.

3.5.6 Temporal Dynamics

State, attention, and policy do not update at the same rate. State is reactive; it changes when inputs change. Attention reallocates fluidly. Policy updates slowly, by design. When these three layers update at mismatched rates, a form of instability emerges that is easy to misread as capability degradation.

An agent that performs inconsistently across similar tasks without any change in its underlying model may be experiencing temporal desynchronization: the policy layer was calibrated for a prior state configuration and is now suppressing information that the current state makes relevant.

The practical implication is straightforward. Before concluding that an agent has degraded, investigate whether its policy layer reflects the current operating environment. Recalibration of policy thresholds, attention weights, or retrieval salience scoring may resolve inconsistencies that appear to be model failures but are actually governance misalignment.

Full treatment of temporal desynchronization, including a worked logistics scenario, appears in the companion paper (Downs, 2026).

3.5.7 Multi-Agent Extension

In multi-agent systems, each agent has its own State x Attention x Policy configuration. An orchestrator adds a higher-level policy layer above all of them.

The orchestrator's policy is the most powerful single variable in the system. It shapes what every agent is permitted to act on, what information crosses agent boundaries, what gets committed to long-term memory. It governs the operative state of every agent it coordinates.

This creates a second-order gating risk. The orchestrator's policy layer can develop systematic biases through continuous exposure to the agents it governs. When that happens, every agent's operative state is shaped by a distorted governance frame, and the distortion is invisible at the individual agent level because each agent is functioning correctly within the rules it has been given.

This failure mode, Orchestrator Epistemic Capture, is developed in full in a separate architectural document. The connection to the model presented here is direct: epistemic capture is what happens when the P(t) component of the master governance layer drifts, and no mechanism exists to audit it from outside its own epistemic frame. Understanding operative cognition at the individual agent level is the foundation for understanding why governance failures at the orchestrator level are difficult to detect and consequential when they occur.

---

4. State and Observability

4.1 The Logging Question

A common question in agent development: if state updates at every step, should every update be logged?

The question reveals a conceptual confusion between two distinct concerns. State management enables the agent to reason effectively. Observability enables humans to understand, debug, and evaluate agent behavior. These concerns have different optimization targets.

Concern	Optimized For	Update Frequency	Persistence
State	Agent reasoning speed and relevance	Every step	Task-scoped
Observability	Debugging, evaluation, compliance	Selective	Long-term
Operative-state monitoring	Gating failure detection	Decision points	Medium-term

Section 3.5 introduces the third row. Operative-state monitoring is instrumentation specifically designed to detect gating failures by measuring the gap between what was retrieved and what actually influenced decisions.

4.2 Modern Observability Standards

The emerging consensus in AI observability, reflected in OpenTelemetry's GenAI semantic conventions (OpenTelemetry, 2025) and commercial platforms including Datadog, Langfuse, and Arize, emphasizes comprehensive tracing of complete execution flows, structured telemetry using standardized schemas for metrics, events, logs, and traces, intelligent sampling to manage volume, and cost and performance tracking across token usage and latency.

This represents a shift from selective logging toward comprehensive instrumentation with smart analysis. The operative-state monitoring layer described in this paper fits naturally within this approach: it adds decision-reference tracking as a first-class observability concern alongside retrieval and tool-call logging.

4.3 Reconciling State and Observability

State is the agent's working memory. It should be lean, containing only task-relevant information.

Observability is the human operator's window into agent behavior. Observability infrastructure should capture comprehensive telemetry, then filter and analyze as needed.

Operative-state monitoring is the layer between them. It tracks the latent-to-operative transition, confirming not just that information entered state, but that it influenced downstream decisions.

Conflating these concerns leads to specific failure patterns. Making state serve observability needs produces bloated schemas that slow reasoning. Limiting observability to state contents misses gating failure signals entirely. Each concern requires its own instrumentation strategy.

4.4 Practical Recommendation

For production systems, the recommended approach proceeds in five steps.

Design state for reasoning, including only information the agent needs for current decisions. Instrument comprehensively using OpenTelemetry or equivalent to capture full traces. Add decision-reference tracking by logging which state fields were referenced at each decision point; this surfaces the latent/operative gap. Apply intelligent sampling strategies to manage storage and cost. Build dashboards and alerts focused on decision-relevant events, including gating failure indicators where high-value fields are available but consistently unreferenced at decision points.

---

5. A Taxonomy of State Updates

This section presents a novel taxonomy classifying state updates by their observability requirements. The taxonomy is the author's contribution, synthesizing insights from CoALA's memory taxonomy, production experience, and observability best practices. It is a proposed practitioner framework, not an empirically validated classification.

5.1 The Four Categories

Category 1: Ephemeral Reasoning

Definition: Internal, high-frequency state changes that exist only to reach the next reasoning step.

Examples: Intermediate calculations during planning, draft thoughts before final formulation, temporary variables in multi-step reasoning, token-level generation states.

Observability Policy: Do not persist to logs. These updates have high volume and low long-term value. They may be captured in debug modes but should not appear in production telemetry.

Rationale: Ephemeral reasoning represents the scratchpad activity that enables reasoning but need not be preserved. Following CoALA's distinction between working memory and long-term memory, these updates belong in neither the observability record nor the memory store.

Gating note: Ephemeral reasoning exists almost entirely in latent state. It is the raw material from which operative state is formed. The intermediate steps themselves rarely drive final decisions directly.

Category 2: Decision-Relevant Updates

Definition: State changes that affect which action the agent takes next.

Examples: Plan modifications or revisions, branch selection in conditional logic, retry decisions after failures, confidence threshold crossings, tool selection decisions.

Observability Policy: Log as structured events capturing the decision, its rationale, and the resulting action.

Example Event Structure:

```
{
  "event_type": "decision",
  "decision_id": "d-12345",
  "category": "plan_revision",
  "reason": "Initial search returned no results",
  "action_selected": "broaden_query",
  "state_fields_referenced": ["search_results",
"research_question"],
  "timestamp": "2025-12-23T10:15:30Z"
}
```

Rationale: Decision events form the audit trail explaining why the agent behaved as it did. They are essential for debugging, evaluation, and compliance.

Gating note: The `state_fields_referenced` field surfaces the operative state at each decision point. By logging which fields were actually consulted, engineers gain direct visibility into what was available versus what was used. This is the instrumentation that makes gating failures detectable.

Category 3: External Interaction Updates

Definition: State changes resulting from interaction with systems outside the agent.

Examples: Tool calls and their responses, API requests and results, database queries and returned data, user inputs and agent outputs, file system operations.

Observability Policy: Always log with full request/response details, or hashes and summaries for large payloads.

Rationale: External interactions are the agent's interface with the world. They must be logged for debugging, cost tracking, compliance verification, and reproducibility.

Category 4: Memory-Qualifying Updates

Definition: State changes that should persist beyond the current task, candidates for long-term memory.

Examples: User preferences discovered during interaction, successful strategies worth preserving, error patterns indicating systematic issues, corrections or feedback from users, domain knowledge extracted during execution.

Observability Policy: Log the event and write to long-term memory storage.

Example Event Structure:

```
{
  "event_type": "memory_formation",
  "memory_type": "semantic",
  "content": "User prefers concise responses without extensive
caveats",
  "confidence": 0.85,
  "source_task": "task-67890",
  "operative_at_formation": true,
  "timestamp": "2025-12-23T10:20:00Z"
}
```

Rationale: Memory-qualifying updates represent the agent's capacity to improve through experience. They bridge working memory and long-term memory.

Gating note: The `operative_at_formation` flag distinguishes memories formed from operative state from those formed from latent state. A memory formed from latent state carries lower reliability as a learned heuristic, because the information that prompted it may not have been genuinely influencing decision-making at the time of formation.

5.2 Summary Table

Category	Log?	Persist to Memory?	Frequency	Gating Layer
Ephemeral Reasoning	No	No	High	Latent
Decision-Relevant	Yes, with fields referenced	No	Medium	Operative
External Interaction	Yes, full detail	No	Medium	Boundary
Memory-Qualifying	Yes, with operative flag	Yes	Low	Operative preferred

5.3 Applying the Taxonomy

When implementing state updates, ask four questions in sequence.

Does this change affect what action comes next? Decision-Relevant. Does this involve an external system? External Interaction. Should this be remembered for future tasks? Memory-Qualifying. None of the above? Ephemeral Reasoning.

Then ask the additional question introduced in Section 3.5: was this information operative, meaning actively referenced at the decision point, or merely latent? This determines reliability weighting for memory-qualifying events and flags potential gating failures where critical information was available but unreferenced at decision points.

5.4 Relationship to OpenTelemetry Standards

This taxonomy aligns with emerging OpenTelemetry GenAI semantic conventions. Ephemeral reasoning maps to detailed spans captured in debug mode but excluded from production traces. Decision-relevant updates map to agent decision spans extended here with `state_fields_referenced`. External interactions map to tool and function call spans with input/output attributes. Memory-qualifying updates map to learning events with memory system instrumentation, extended here with `operative_at_formation`.

The taxonomy provides conceptual clarity. OpenTelemetry provides implementation standards. They are complementary.

---

6. Multi-Agent State Coordination

6.1 The Coordination Challenge

When multiple agents collaborate, state management complexity increases in ways that individual agent design patterns do not anticipate. The fundamental question is how agents share information while maintaining coherent individual reasoning.

The multi-agent systems literature (Guo et al., 2024; Wooldridge, 2009) identifies several coordination models, each with distinct implications for state architecture.

Model	Description	State Implications
Centralized	Single coordinator dispatches to workers	Coordinator maintains global state; workers have local state
Decentralized	Agents communicate peer-to-peer	Each agent maintains own state; explicit message passing
Hierarchical	Teams with local supervisors	Layered state with team-level and global-level views
Blackboard	Shared workspace all agents access	Single shared state object with concurrent access patterns

In governed multi-agent systems, the orchestrator's policy layer shapes what information from shared state becomes operative for each agent. State coordination and operative cognition are therefore inseparable concerns at the multi-agent level.

6.2 Shared State Architecture

LangGraph and similar frameworks typically implement a shared state model where all agents read from and write to a common state object. This approach, aligned with CoALA's working memory concept extended to multiple agents, requires careful schema design.

```
class MultiAgentState(TypedDict):
    # Global coordination
    overall_goal: str
    current_phase: str
    agents_completed: list[str]

    # Agent-specific sections
    researcher_output: Optional[ResearchResult]
    writer_output: Optional[str]
    reviewer_feedback: Optional[ReviewResult]

    # Shared artifacts
    final_deliverable: Optional[str]
    revision_count: int
```

6.3 State Contracts

For reliable multi-agent coordination, this paper proposes the concept of state contracts: explicit specifications of what each agent reads and writes, along with the preconditions required before an agent can execute and the postconditions it must satisfy when complete.

```
Agent: Writer
Reads: research_findings, outline, style_preferences
Writes: draft_content, writing_status
Preconditions:
  - research_findings is not empty
  - outline contains at least 3 sections
Postconditions:
```

```
  - draft_content is non-empty string
  - writing_status in ["complete", "needs_revision"]
```

State contracts enable static validation before runtime, clear debugging when handoffs fail by identifying which preconditions were violated, executable documentation, and isolated agent testing. They also define the expected operative surface: the fields an agent is supposed to act on. When decision-reference logging reveals an agent consistently not referencing fields listed in its contract's Reads, that gap is a gating failure signal worth investigating.

6.4 Coordination Patterns

Three coordination patterns have emerged as practical standards for multi-agent state management.

The **Supervisor Pattern** maintains a central supervisor agent that holds global state and dispatches tasks to worker agents. Workers return results to the supervisor, which updates shared state and determines next steps.

```
Supervisor ──► Worker A ──► Supervisor ──► Worker B ──►
Supervisor
     │                          │                          │
     └── updates state ──────────┴── updates state ──────────┘
```

The **Pipeline Pattern** executes agents in sequence, each consuming predecessor output. State flows linearly with clear handoff points.

```
Agent A ──► Agent B ──► Agent C ──► Final Output
   │            │            │
   └── state ───┴── state ───┘
```

The **Collaborative Pattern** operates multiple agents on a shared workspace simultaneously. This requires conflict resolution strategies: last-write-wins for simplicity at the cost of information loss, merge functions with custom logic combining concurrent updates, or locking for exclusive state section access at the cost of parallelism.

6.5 Validation at Handoffs

Multi-agent systems require validation at agent boundaries. The following pattern prevents cascading failures where one agent's incomplete output corrupts downstream agents.

```
def validate_handoff(state: MultiAgentState, from_agent: str,
to_agent: str) -> bool:
    """Validate state before agent handoff."""

    contract = get_contract(to_agent)

    for condition in contract.preconditions:
        if not evaluate_condition(condition, state):
            raise HandoffError(
                f"Precondition failed for {to_agent}:
{condition}"
            )

    for field in contract.reads:
        if field not in state or state[field] is None:
            raise HandoffError(
                f"Required field missing for {to_agent}:
{field}"
            )

    return True
```

Boundary validation defines the outer boundary of what is available to the receiving agent. It does not guarantee that available information will become operative. Boundary validation and operative state monitoring address different failure surfaces and are complementary rather than substitutable.

7. Production Deployment Guidelines

7.1 State Schema Design Process

A systematic process for state schema design proceeds through five steps.

Step 1: Task Analysis. Identify what the agent is trying to accomplish, what information it needs at each step, and what decisions must be traceable.

Step 2: Information Inventory. Enumerate inputs from users or upstream systems, retrieved or generated intermediate results, decisions and their rationale, and final outputs and deliverables.

Step 3: Schema Definition. Define a TypedDict or Pydantic schema. Annotate all fields with types. Identify optional versus required fields. Specify reducers for concurrent updates.

Step 4: Validation Rules. Define preconditions for each processing step, postconditions confirming step completion, and invariants that must always hold.

Step 5: Observability Integration. Map state fields to telemetry events, define what triggers logging, configure sampling and retention, and add decision-reference tracking for gating failure detection.

7.2 Design Checklist

Before implementing an agent, verify the following.

Core Structure:

- [ ] Goal/objective field defined
- [ ] Progress tracking mechanism included
- [ ] All inputs explicitly represented
- [ ] Intermediate results captured
- [ ] Final output field specified

Technical Requirements:

- [ ] Schema uses TypedDict, dataclass, or Pydantic
- [ ] All fields have type annotations

- [ ] Optional fields properly marked
- [ ] Collections typed as list[T] or dict[K, V]
- [ ] State serializable to JSON

Observability:

- [ ] Logging category assigned to each update type
- [ ] External interactions always logged
- [ ] Decision events captured with rationale
- [ ] Decision events include `state_fields_referenced`
- [ ] Memory-qualifying updates route to persistence
- [ ] Memory-qualifying updates include `operative_at_formation` flag

Validation:

- [ ] Preconditions defined for each step
- [ ] Postconditions verify step completion
- [ ] Error handling preserves valid state

Gating Failure Detection:

- [ ] Contract Reads fields cross-referenced against `state_fields_referenced` logs
- [ ] Alerts defined for fields in Reads consistently absent from decision references
- [ ] Policy layer reviewed periodically for alignment with current state configuration

7.3 State Persistence Strategies

Three persistence strategies are available, each with distinct tradeoffs.

Checkpointing saves complete state snapshots periodically. LangGraph provides built-in checkpointers for Postgres, Redis, SQLite, and custom backends. This enables recovery from failures, human-in-the-loop pause and resume, and time-travel debugging.

Event Sourcing stores the sequence of state-changing events. State is reconstructed by replaying those events. This provides a complete audit trail and a natural fit for the logging taxonomy.

Hybrid Approach combines periodic checkpoints for fast recovery with a continuous event log for auditability. This balances performance with completeness and is the recommended pattern for most production systems.

7.4 From Prototype to Production

Aspect	Prototype	Production
State Storage	In-memory dict	Database with transactions
Validation	Optional, print statements	Required at every transition
Logging	Basic logging	Structured telemetry via OpenTelemetry
Serialization	json.dumps()	Versioned schemas with migration
Error Handling	Exceptions propagate	Graceful degradation, state preservation
Concurrency	Single-threaded	Locking or event sourcing
Gating Monitoring	None	Decision-reference tracking, operative-state alerts

7.5 Observability Integration

For production observability, the recommended instrumentation uses the OpenTelemetry SDK with GenAI semantic conventions. All LLM calls should be instrumented with model, tokens, and latency. All tool calls should be instrumented with inputs and outputs. Agent decision points should produce dedicated spans. Those spans should include `state_fields_referenced`.

Dashboards should surface success and failure rates by agent and task type, latency distributions across execution steps, token usage and cost attribution, error categorization and frequency, and gating failure rates defined as fields available but not referenced at decision points.

Alerting should trigger on elevated error rates, latency threshold breaches, unusual token consumption, state validation failures, and contract fields consistently absent from decision references.

---

8. Discussion

8.1 Summary of Contributions

This paper has developed a practitioner-oriented framework for agent state management grounded in the theoretical foundations established by CoALA (Sumers et al., 2024). The primary contributions are five.

First, an operational interpretation of working memory that translates CoALA's theoretical concepts into software engineering terms with explicit schema design guidance.

Second, a logging taxonomy providing a novel four-category classification with observability strategy guidance, extended in this edition with gating-aware instrumentation fields.

Third, state contracts for multi-agent systems: a proposed mechanism for specifying and validating agent interactions in multi-agent architectures.

Fourth, design patterns and checklists as practical tools for state schema design, validation, and production deployment.

Fifth, the latent versus operative state model: a structural extension of the state-primacy principle that distinguishes between information present in state and information actively shaping behavior, with direct implications for instrumentation design, debugging protocol, and multi-agent governance.

8.2 Limitations

Several limitations deserve acknowledgment.

The logging taxonomy, design recommendations, and latent/operative state model are based on synthesis of existing work and practical experience, not systematic empirical study. Validation against diverse agent architectures would strengthen these contributions.

The framework assumes LLM-based agents. Applicability to other agent architectures is not addressed.

The agentic AI tooling landscape changes rapidly. Specific recommendations regarding LangGraph, OpenTelemetry, and other tools may require updates as these projects evolve.

Performance characteristics of different state management approaches at scale are not analyzed. The overhead of validation and comprehensive logging in high-throughput production environments warrants dedicated investigation.

State may contain sensitive information. Encryption, access control, and privacy-preserving state management are not addressed here.

The `state_fields_referenced` instrumentation proposed in this paper requires custom implementation. No standard framework currently surfaces this signal automatically. This remains an open engineering gap.

8.3 Future Directions

Empirical validation of the logging taxonomy and gating failure model across different agent types, tasks, and scales represents the most significant near-term research need.

Standardization of state schemas and contracts to enable interoperability between agent frameworks, including standardized operative-state telemetry, would meaningfully advance the field.

Automated gating detection tools that analyze decision-reference logs to surface latent-state suppression patterns would remove the burden of manual investigation from practitioners.

Formal mathematical specification of the attention and policy functions in $C(t) \approx P(t)(A(t)(S(t)))$ would enable quantitative analysis of gating failure conditions.

This direction is developed further in the companion theoretical paper (Downs, 2026).

Privacy-preserving state management techniques that maintain useful working memory while protecting sensitive information represent a separate but urgent direction as agentic systems enter regulated domains.

8.4 Conclusion

Agent state management is foundational to building effective LLM-based agents. The theoretical groundwork laid by CoALA and classical cognitive architectures provides essential conceptual clarity. This paper has attempted to bridge that theory to practice through frameworks, patterns, and engineering guidance.

The revised edition extends that bridge in one important direction: from what information enters state to whether that information ever influences behavior. That extension matters because the most common misdiagnosis in production agent systems is treating a gating failure as a retrieval failure. Adding more memory to a system with a policy-layer failure wastes resources. Improving retrieval in a system with an attention weighting problem addresses the wrong layer. The distinction between latent and operative state makes the right diagnosis possible.

Two core insights run through this paper and build on each other.

State schema design is architecture, not implementation detail. The choice of what information flows through an agent system determines what that system can reason about.

State presence is not state operability. The choice of how information is weighted, prioritized, and permitted to influence action determines what the system actually does.

Both insights are necessary. Neither is sufficient alone. Together, they define the practical frontier of agent state management as the field matures.

References

Anderson, J. R. (1996). ACT: A simple theory of complex cognition. *American Psychologist, 51*(4), 355-365.

Anderson, J. R., & Lebiere, C. (1998). *The atomic components of thought.* Lawrence Erlbaum Associates.

Downs, C. (2026). *The Cognitive Horizon of an Agent: A Structural Model of Operative Cognition in Agentic Systems.* Professor Bone Lab Working Paper.

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., & Zhang, X. (2024). Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680.*

Kotseruba, I., & Tsotsos, J. K. (2020). 40 years of cognitive architectures: Core cognitive abilities and practical applications. *Artificial Intelligence Review, 53*(1), 17-94.

Laird, J. E. (2012). *The Soar cognitive architecture.* MIT Press.

Laird, J. E. (2022). Introduction to Soar. *arXiv preprint arXiv:2205.03854.*

Laird, J. E., Newell, A., & Rosenbloom, P. S. (1987). SOAR: An architecture for general intelligence. *Artificial Intelligence, 33*(1), 1-64.

LangChain. (2024). LangGraph documentation. https://langchain-ai.github.io/langgraph/

OpenTelemetry. (2025, March 12). AI agent observability: Evolving standards and best practices. *OpenTelemetry Blog.* https://opentelemetry.io/blog/2025/ai-agent-observability/

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. *Proceedings of UIST 2023.*

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *NeurIPS 2023.* arXiv:2303.11366.

Sumers, T. R., Yao, S., Narasimhan, K., & Griffiths, T. L. (2024). Cognitive architectures for language agents. *Transactions on Machine Learning Research.*

Wang, L., Ma, C., Feng, X., et al. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science, 18*(6), 186345.

Wooldridge, M. (2009). *An introduction to multiagent systems* (2nd ed.). Wiley.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR 2023.*

Appendix A: State Schema Examples

A.1 Research Agent State

```
from typing import TypedDict, Optional, Annotated
import operator

class Source(TypedDict):
    """A source document retrieved during research."""
    title: str
    url: str
    snippet: str
    relevance_score: float

class ResearchAgentState(TypedDict):
    """
    State schema for a research agent.

    Follows CoALA working memory principles:
    - Contains only task-relevant information
    - Structured for agent reasoning
    - Supports progress tracking and decision tracing
```

```
    Fields marked [OPERATIVE-TRACK] are high-value candidates
    for decision-reference logging.
    """

    # Task context
    research_question: str         # [OPERATIVE-TRACK]
    constraints: list[str]

    # Progress tracking
    steps_completed: Annotated[list[str], operator.add]
    current_phase: str  # "planning" | "searching" | "analyzing"
| "synthesizing"

    # Retrieved information
    search_queries_tried: list[str]
    sources_found: list[Source]
    sources_selected: list[Source]  # [OPERATIVE-TRACK]

    # Analysis
    key_findings: list[str]         # [OPERATIVE-TRACK]
    contradictions_noted: list[str]
    gaps_identified: list[str]

    # Output
    draft_answer: Optional[str]
    final_answer: Optional[str]
    confidence: Optional[float]

    # Error handling
    errors_encountered: list[dict]
    retry_count: int
```

A.2 Multi-Agent Collaborative State

```
from typing import TypedDict, Optional, Literal
from enum import Enum

class AgentStatus(str, Enum):
    PENDING = "pending"
```

```
    RUNNING = "running"
    COMPLETE = "complete"
    FAILED = "failed"
    BLOCKED = "blocked"

class CollaborativeState(TypedDict):
    """
    State schema for multi-agent collaboration.

    Implements shared state pattern with agent-specific sections
    and coordination metadata.
    """

    # Global coordination
    task_id: str
    overall_goal: str
    deadline: Optional[str]

    # Phase management
    current_phase: Literal["research", "drafting", "review",
"finalization"]
    phases_completed: list[str]

    # Agent status tracking
    researcher_status: AgentStatus
    writer_status: AgentStatus
    reviewer_status: AgentStatus

    # Agent outputs structured for handoff
    research_output: Optional[dict]
    draft_output: Optional[str]
    review_output: Optional[dict]

    # Iteration tracking
    revision_count: int
    max_revisions: int
    revision_history: list[dict]

    # Final deliverable
```

```
    final_output: Optional[str]
    approval_status: Literal["pending", "approved", "rejected"]
```

Appendix B: Validation Patterns

B.1 Step Validation

```
from typing import Callable
from dataclasses import dataclass

@dataclass
class StepValidator:
    """Validates state before and after agent steps."""

    preconditions: dict[str, Callable[[dict], bool]]
    postconditions: dict[str, Callable[[dict], bool]]

    def validate_preconditions(self, state: dict, step: str) ->
list[str]:
        violations = []
        if step not in self.preconditions:
            return violations
        for name, check in self.preconditions[step].items():
            if not check(state):
                violations.append(f"Precondition '{name}' failed
for step '{step}'")
        return violations

    def validate_postconditions(self, state: dict, step: str) ->
list[str]:
        violations = []
        if step not in self.postconditions:
            return violations
```

```
        for name, check in self.postconditions[step].items():
            if not check(state):
                violations.append(f"Postcondition '{name}'
failed for step '{step}'")
        return violations

research_validator = StepValidator(
    preconditions={
        "search": {"has_question": lambda s:
bool(s.get("research_question"))},
        "analyze": {"has_sources": lambda s:
len(s.get("sources_selected", [])) > 0},
        "synthesize": {"has_findings": lambda s:
len(s.get("key_findings", [])) > 0},
    },
    postconditions={
        "search": {"found_sources": lambda s:
len(s.get("sources_found", [])) > 0},
        "analyze": {"extracted_findings": lambda s:
len(s.get("key_findings", [])) > 0},
        "synthesize": {"produced_answer": lambda s:
s.get("final_answer") is not None},
    }
)
```

B.2 Contract-Based Handoff Validation

```
@dataclass
class AgentContract:
    """Specifies an agent's state interface."""

    agent_name: str
    reads: list[str]
    writes: list[str]
    requires_agents: list[str]

    def validate_can_start(self, state: dict, completed_agents:
list[str]) -> bool:
        for required in self.requires_agents:
```

```
            if required not in completed_agents:
                raise ContractViolation(
                    f"{self.agent_name} requires {required} to
complete first"
                )
        for field in self.reads:
            if field not in state or state[field] is None:
                raise ContractViolation(
                    f"{self.agent_name} requires field '{field}'
but it is missing"
                )
        return True

    def validate_completed(self, state: dict) -> bool:
        for field in self.writes:
            if field not in state or state[field] is None:
                raise ContractViolation(
                    f"{self.agent_name} should have produced
'{field}' but did not"
                )
        return True

CONTRACTS = {
    "researcher": AgentContract(
        agent_name="researcher",
        reads=["research_question"],
        writes=["research_output"],
        requires_agents=[],
    ),
    "writer": AgentContract(
        agent_name="writer",
        reads=["research_output", "overall_goal"],
        writes=["draft_output"],
        requires_agents=["researcher"],
    ),
    "reviewer": AgentContract(
        agent_name="reviewer",
        reads=["draft_output", "research_output"],
        writes=["review_output"],
```

```
        requires_agents=["writer"],
    ),
}
```

Appendix C: Logging Implementation

C.1 Structured Event Logging with Operative-State Tracking

```
import json
from datetime import datetime
from enum import Enum
from typing import Any, Optional
from dataclasses import dataclass, asdict, field

class UpdateCategory(str, Enum):
    EPHEMERAL = "ephemeral"
    DECISION = "decision"
    EXTERNAL = "external_interaction"
    MEMORY = "memory_qualifying"

@dataclass
class StateEvent:
    """Structured event for state changes."""

    event_id: str
    timestamp: str
    category: UpdateCategory
    event_type: str
    agent_name: str
    details: dict[str, Any]
    state_fields_referenced: Optional[list[str]] =
field(default=None)
    operative_at_formation: Optional[bool] = field(default=None)
```

```
    def to_json(self) -> str:
        return json.dumps(asdict(self), default=str)

class StateLogger:
    """Logger implementing the four-category taxonomy with
operative-state tracking."""

    def __init__(self, logger, memory_store=None):
        self.logger = logger
        self.memory_store = memory_store

    def log_update(self, category: UpdateCategory, event_type:
str,
                   agent_name: str, details: dict,
                   state_fields_referenced: Optional[list[str]]
= None,
                   operative_at_formation: Optional[bool] =
None) -> None:

        if category == UpdateCategory.EPHEMERAL:
            return

        event = StateEvent(
            event_id=self._generate_id(),
            timestamp=datetime.utcnow().isoformat(),
            category=category,
            event_type=event_type,
            agent_name=agent_name,
            details=details,
            state_fields_referenced=state_fields_referenced,
            operative_at_formation=operative_at_formation,
        )

        self.logger.info(event.to_json())

        if category == UpdateCategory.MEMORY and
self.memory_store:
            self.memory_store.save(event)
```

```
    def log_decision(self, agent_name: str, decision_type: str,
                     reason: str, action_selected: str,
                     state_fields_referenced: list[str]) ->
None:
        """Log a decision event with operative-state reference
tracking."""
        self.log_update(
            category=UpdateCategory.DECISION,
            event_type="decision",
            agent_name=agent_name,
            details={
                "decision_type": decision_type,
                "reason": reason,
                "action_selected": action_selected,
            },
            state_fields_referenced=state_fields_referenced,
        )

    def log_tool_call(self, agent_name: str, tool_name: str,
                      inputs: dict, outputs: dict, latency_ms:
float) -> None:
        self.log_update(
            category=UpdateCategory.EXTERNAL,
            event_type="tool_call",
            agent_name=agent_name,
            details={
                "tool_name": tool_name,
                "inputs": inputs,
                "outputs": outputs,
                "latency_ms": latency_ms,
            }
        )
```

Professor Bone Lab
Draft v3, Revised February 2026
Classification: Working paper, empirical validation and peer review invited

This work is licensed under CC BY 4.0. *It builds substantially on the CoALA framework developed by Sumers, Yao, Narasimhan, and Griffiths. The author thanks the developers of LangGraph, OpenTelemetry, and the broader agentic AI community for the tools and standards that informed this work.*